

C's grunder lite mer metodiskt

- Identifierare och nyckelord
- Heltals och flyttalstyper
- Typomvandlingar
- `enum`
- Mera om `printf` och `scanf`
- Uttryck och operatorer
- Satser

Identifierare och nyckelord

- Namn som man hittar på själv kallas identifierare
 - de är namn på variabler, konstanter, funktioner
 - börjar på bokstav (A–Z, a–z), följd av bokstäver, siffror och `_`
 - Exempel: `i`, `Summa`, `min_egen2`, `printf`, `main`, `f_to_c`
- Nyckelord (reserverade ord)
 - får bara användas till avsett ändamål
 - Exempel: `int`, `if`, `for`, `while`, `float`

Heltalstyper

► Vi har redan sett `int` och `char`

Typ	Omfång (garanterat enligt ANSI)
<code>char</code>	0 till 127
<code>short int</code>	-32767 till 32767
<code>int</code>	-32767 till 32767 (ofta numera som <code>long</code>)
<code>long int</code>	-2147483647 till 2147483647

Heltalstyper (forts)

- ▶ Ofta har variabler större omfång
 - `char` är ofta `-128` till `127` *eller* `0` till `255`
 - `int` är ofta lika stor som `long int`
- ▶ Om man går utanför gränserna är det odefinierat vad som händer, oftast varvar det runt
- ▶ Processorns beteende bruka lysa igenom
- ▶ Talen lagras internt som 1/0 mönster, *binärt*

Exempel, heltalstyper

```
int i;
char c;
long l;
scanf("%d", &i);
l = i;          /* alltid OK */
c = i;          /* OK om i inte är för stort */
i++;            /* OK om i inte blir för stort */
l = i / 67;     /* Heltalsdivision, strunta i rest */
```

Flyttal

- Lagras i två delar
 - exponent, talar om var decimalkommat skall vara
 - mantissa, en radde decimaler
 - (i själva verket lagras både exponent och mantissa binärt)
- Tre storlekar: `float`, `double` och `long double`
 - ger olika många decimalers precision och olika omfång på exponenten

Exempel, heltal och flyttal

► Konvertering sker automatiskt

- decimalerna *huggs av* vid konvertering hel- till flyttal

```
int i, j = 56;
```

```
float f, g = 78.98;
```

```
f = 3.4E6;          /* 3.4 miljoner (10 upphöjt till 6) */
```

```
j = f;             /* går bra om int är så stor */
```

```
j = g;             /* j får värdet 78 */
```

```
f = i / j;          /* heltalsdivision! */
```

```
f = i / 3.0;        /* flyttalsdivision */
```

```
f = (float)i / j;   /* uttrycklig konvertering */
```

Uppräkningstyper, enum

- Egna påhittade värden
- Tolkas och används som heltal

```
enum day { mon, tue, wed, thu, fri, sat, sun };  
enum day d;  /* mon=0, tue=1, wed=2 o.s.v */  
  
d = mon;  
  
/* ... */  
  
if (d == wed)  
    printf("%d", d);  
  
d++;
```


Mer om printf

► Fler formatkoder

- Användaren *måste* själv använda rätt kod

`%d` heltal decimalt, äver för char och short

`%ld` long int

`%c` teckenkod, skriv ut motsvarande tecken

`%f` flyttal, både float och double

`%e` flyttal, med exponentnotation

`%s` skriv ut textsträng

Många fler finns, se sid 57 i boken

Mer om scanf

- ▶ `scanf` kollar inte att talet som läses in får plats i den variabel som läses till!
 - använd inte `scanf` i *riktiga* program
- ▶ `scanf` är mer petig med formatkoderna

`%hd` `short int`

`%lf` `double`

Med flera, se sid 60

Uttryck

- ▶ Ett uttryck är något som har ett värde
 - en konstant `3578`, `'A'`, `78.99`
 - en variabel `summa`, `i`, `p`
 - en beräkning `i * 89`, `p + 1.78`
 - ett funktionsanrop `f_to_c(98.7)`,
`scanf("%d", &i)`
 - alla kombinationer av dessa
- ▶ Överallt där ett uttryck kan användas, kan det vara hur komplicerat som helst

Operatorer

- Operatorer beräknar nya värden av uttryck
 - en del vill ha två operander $i / 3$
 - en del vill bara ha en operand $i++$
 - en del kan vara antingen eller $i - 3$ $-i$
- Alla operatorer har *prioritet*
 - t ex multiplikation går före addition
 - prioritet kan ändras med parenteser
 - prioritetsordningen mellan operatorer är den ”naturliga” (men det finns ca 20 nivåer)

Aritmetiska operatörer

+ - addition, subtraktion

* / multiplikation division

(OBS heltalsdivision om båda sidor är heltal)

% restoperator, t ex 34 % 10 ger 4

```
int i, j = 3; float f;
```

```
i = 4 + j * 2; /* i får värdet 10 */
```

```
i = (4 + j) * 2; /* i får värdet 14 */
```

```
f = 1 / j; /* f får värdet 0.0 */
```

```
f = 1.0 / j; /* f får värdet 0.333333... */
```

Öknings- och minskningsoperatorer

- Att öka och minska med 1 är vanligt

```
int i, j=0, k=0;  
i = k++; /* i får k's gamla värde 0, k ökar */  
i = ++j; /* i får j's nya värde 1, j ökar */  
i++;    /* oftast bryr man sig inte om värdet */  
i--;    /* samma regler för -- */
```

Exempel

```
/* beräkning av antal blanka i en text */
char text[100];
int ant_blank, i;

/* text fylls på med en textsträng på något sätt */

ant_blank = 0;
i = 0;
while (text[i] != '\0')
    if (text[i++] == ' ')
        ++ant_blank;
printf("Antal blanka: %d\n", ant_blank);
```

Jämförelseoperatorer

< <= > >= för både flyt och heltal
== != likhet och olikhet

Alla jämförelseoperatorer ger värde

1 för sant

0 för falskt

```
if (villkor)   /* if tolkar 0 som falskt, allt annat som sant */  
    sats;
```

if (i != 0) blir alltså samma som if (i)

En del C-programmerare tycker: ju kortare ju vackrare...

Logiska operatorer

► Kombinerar logiska uttryck till sant/falskt (0/1)

&&, ||, ! beräknar *och*, *eller*, *icke*

```
int c; char cc[10]; int n = 0;
```

```
c = getchar();
```

```
if (c < '0' || c > '9') printf("Inte siffra\n");
```

```
while (n < 9 && (c = getchar()) != EOF)
```

```
    cc[n++] = c;          /* fyll cc */
```

```
cc[n] = '\0';
```

```
printf("%s", cc);
```

Tilldelningsoperatorer

- Uttryck av typen $i = i + 10$ är vanliga
 - ett förenklat skrivsätt finns, speciellt bra om i är komplicerat

```
int i;          /* fahrenheit/celsius-tabell */  
for (i = -50; i <= 200; i += 10)  
    printf("%5d %7.2f", i, f_to_c(i));
```

De flesta varianter finns:

`+=` `-=` `*=` `/=` med flera

sizeof-operatorn

- ▶ Används för att beräkna storleken av en variabel eller typ
 - ger storleken i minnesceller (bytes)
 - `sizeof (char)` ger 1, `sizeof (long)` ger (oftast) 4
- ▶ Används vid *dynamisk minnesallokering* (kommer senare)
 - `100 * sizeof(int)` ger oss ett minnesmått för hundra heltal

Satser

- ▶ *Satser* är de ”som utför något”
 - ett uttryck med ett semikolon: `i = 45 * j; i++;`
 - en `if`-sats: `if (i > 0) j = 75;`
 - en `while`-sats
 - en sammansatt sats: `{ i = 14; j = 54; }`
- ▶ Vi har gått igenom `if`, `for`, `while` och uttryckssatser
 - vi har `do/while`, `switch` och hoppsatser kvar

do/while-satsen

- Precis som while, fast med testet vid slutet

```
int c;  
do  
{  
    c = getchar();  
    /* gör nåt */  
} while (c != 'Q');
```

switch-satsen

► Som if, fast med flerval

```
int c;
do {
    c = getchar();
    switch(c)
    {
        case 'a': /* gör a-jobb */; break;
        case 'B':
        case 'b': /* gör b-jobb */; break
        case 'Q': printf("Hej då\n"); break;
        default:  printf("Okänt kommando\n");
    }
} while (c != Q);
```

Några detaljer

- ▶ `switch`-satsen har vissa egenskaper
 - `switch`-värdet måste vara ett heltal
 - `case`-värdena måste vara konstanter
 - varje avdelning avslutas med `break`; annars ramlar man igenom till nästa `case`
 - om ingen `case` matchar, görs `default` (om den finns)
 - avdelningarna kan komma i vilken ordning som helst, men `case`-värdena måste vara unika

return- break- och goto-satsen

- ▶ `return` avbryter funktionen och returnerar ett eventuellt värde
 - `return` i `main` avbryter programmet
- ▶ `break` avbryter närmast omgivande snurra eller `switch`
- ▶ `goto` hoppar vart som helst inom funktionen
 - skapar dålig och svårläst struktur: undvik!